

МИНОБРНАУКИ РОССИИ

**Федеральное государственное автономное образовательное учреждение
высшего образования**

«Российский технологический университет МИРЭА»

**Институт перспективных технологий и индустриального
программирования**

Кафедра индустриального программирования

ОТЧЕТ

по практическому занятию №4

«Тестирование проекта»

Проект: «Тайм-трекер (учет рабочего времени)»

Поле	Значение
Дисциплина	Программирование корпоративных систем
Вид учебного материала	Практическое занятие
Студент	_____
Группа	ЭФБО-__-24
Преподаватель	Сокунов Дмитрий Антонович
Семестр	2 семестр, 2025-2026 гг.

Москва, 2026

СОДЕРЖАНИЕ

Введение

1 Теоретическая часть

1.1 Анализ предметной области

1.2 Выбор темы и обоснование простоты реализации

1.3 Требования к системе по Вигерсу

2 Технологическая часть

2.1 SRS: назначение и область действия

2.2 Общее описание продукта

2.3 Детальные требования и пользовательские сценарии

2.4 UML-диаграммы

3 Практическая часть

3.1 План пользовательского интерфейса

3.2 План структуры C++ проекта

3.3 План тестирования

3.4 Инструкция по эксплуатации

Заключение

Список использованных источников

Приложения

ВВЕДЕНИЕ

В рамках практического занятия №2 требуется выбрать тему будущего проекта, собрать требования, оформить техническую документацию и подготовить UML-диаграммы. Так как дальнейшая реализация должна выполняться на языке C++, при выборе темы важно учитывать не только интересность идеи, но и сложность объяснения проекта на защите.

Из предложенного списка выбран проект «Тайм-трекер (учет рабочего времени)». Эта тема является одной из самых простых для реализации и защиты: предметная область понятна без специальной подготовки, пользовательские сценарии короткие, а программная часть сводится к работе с задачами, временем, списками и файлами.

Цель работы: сформировать требования и проектную документацию для консольного C++ приложения, которое позволяет фиксировать начало и окончание работы над задачами, хранить журнал времени и формировать простые отчеты.

Для достижения цели были поставлены задачи: выбрать тему проекта; описать предметную область; провести сбор требований; классифицировать требования по Вигерсу; составить SRS-документ; подготовить диаграмму вариантов использования, диаграмму классов и диаграмму последовательности; описать будущую структуру реализации на C++.

1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1.1 Анализ предметной области

Тайм-трекер — это программа для учета рабочего времени. Пользователь создает задачи, запускает таймер при начале работы, останавливает его после завершения и получает журнал записей. По этим записям можно понять, сколько времени было потрачено на конкретную задачу, день или период.

Проблема возникает там, где время учитывается вручную: в блокноте, таблице или по памяти. Такие способы быстро приводят к пропущенным записям, ошибкам в длительности и невозможности быстро получить итоговый отчет. Даже простая локальная программа решает эту проблему, потому что хранит записи структурированно и считает длительность автоматически.

Для учебного проекта предметная область удобна тем, что все сущности легко объяснить. Есть задача, есть запись времени, есть отчет. Не требуется сеть, база данных, графический интерфейс или сложная математика. В дальнейшем проект можно расширять, но минимальная версия уже будет полезной и проверяемой.

1.2 Выбор темы и обоснование простоты реализации

Тема	Сложность	Что нужно реализовать	Комментарий
Тайм-трекер	Низкая	Файлы, время, списки, меню	Понятная предметная область, легко показать на защите
Генератор лабиринтов	Средняя	Алгоритмы генерации и поиска пути	Нужно объяснять алгоритмы и визуализацию
Визуализатор сортировок	Средняя	Сортировки и отображение анимации	Требуется аккуратная визуальная часть
Клиент-серверный чат	Высокая	Сокеты и шифрование	Сложнее отлаживать и защищать
Прокси-сервер HTTP	Высокая	Сеть, протоколы, кеш	Много технических деталей

По сравнению с другими темами тайм-трекер имеет минимальный порог входа. Для первой версии достаточно консольного меню, структуры данных в памяти и сохранения в CSV-файлы. CSV — это текстовый табличный формат, где строки хранят записи, а значения разделяются символом, например точкой с запятой.

Для защиты тема также удобна. Можно показать простой сценарий: создать задачу, запустить таймер, остановить таймер, открыть отчет. Даже человек, который плохо знает C++, сможет объяснить, что программа хранит список задач и список интервалов времени, а потом суммирует длительности.

1.3 Требования к системе по Вигерсу

Для сбора требований используется традиционный подход: анализ задания, моделирование будущего пользователя и разбор типовых сценариев работы. Требования классифицируются по иерархии Вигерса: бизнес-требования, пользовательские требования, функциональные требования и нефункциональные требования.

Категория	Требование
Бизнес-требования	Сократить потери данных о затраченном времени; дать пользователю понятную картину занятости; подготовить основу для учебного C++ проекта.
Пользовательские требования	Пользователь должен создавать задачи, запускать и останавливать учет времени, просматривать журнал и получать отчет за день или период.
Функциональные требования	Система должна хранить задачи, создавать записи времени, считать длительность, проверять корректность дат, сохранять данные в файлы и загружать их при запуске.
Нефункциональные требования	Программа должна быть простой, локальной, устойчивой к ошибочному вводу, переносимой между компьютерами и достаточно быстрой для сотен или тысяч записей.

Ключевая граница проекта: система не является корпоративной платформой управления проектами. В первой версии она не требует регистрации пользователей, сетевого взаимодействия и базы данных. Это осознанное упрощение, которое делает проект реалистичным для реализации на C++.

2 ТЕХНОЛОГИЧЕСКАЯ ЧАСТЬ

2.1 SRS: назначение и область действия

SRS — это спецификация требований к программному обеспечению. В данном проекте документ фиксирует, что должна делать программа, какие пользователи с ней работают, какие ограничения есть у первой версии и какие данные требуется хранить.

Цель системы: предоставить локальный инструмент для учета рабочего времени по задачам. Область действия первой версии включает консольный интерфейс, хранение данных в файлах, базовые операции с задачами, учет временных интервалов и формирование отчетов.

Термин	Определение
Задача	Работа или активность, по которой пользователь хочет учитывать время.
Запись времени	Интервал с началом, окончанием и привязкой к задаче.
Таймер	Текущая открытая запись времени, у которой уже есть начало, но еще нет окончания.
Отчет	Сводка длительностей по задачам, дням или выбранному периоду.
CSV	Простой текстовый формат для хранения табличных данных.

2.2 Общее описание продукта

Продукт представляет собой консольное приложение на C++. Пользователь взаимодействует с программой через меню: выбирает номер команды, вводит название задачи или дату, получает текстовый результат. Такой интерфейс выбран специально: он проще графического интерфейса и хорошо подходит для учебной реализации.

Основные функции продукта:

- создание, просмотр, переименование и архивирование задач;
- запуск учета времени по выбранной задаче;
- остановка активного таймера с автоматическим расчетом длительности;
- ручное добавление записи времени, если пользователь забыл включить таймер;
- просмотр журнала записей;
- формирование отчета по задачам и по дням;
- сохранение и загрузка данных из файлов.

Характеристика пользователя: обычный студент, фрилансер или сотрудник, которому нужно понять, куда уходит рабочее время. От пользователя не требуется знание C++ или структуры файлов, потому что все операции выполняются через меню.

2.3 Детальные требования и пользовательские сценарии

Сбор требований выполнен через сценарный анализ. Были выделены основные действия пользователя, после чего каждое действие преобразовано в требование. Такой подход проще, чем объемное интервью, и хорошо подходит для учебного проекта с одним типом пользователя.

К од	Требование	При оритет
F R-01	Система должна позволять создать задачу с названием и необязательным описанием.	Выс окий
F R-02	Система должна запускать таймер только для существующей активной задачи.	Выс окий
F R-03	Система должна запрещать запуск второго таймера, пока первый не остановлен.	Выс окий
F R-04	Система должна сохранять время начала и окончания работы.	Выс окий
F R-05	Система должна рассчитывать длительность записи в минутах.	Выс окий
F R-06	Система должна формировать отчет по задачам за выбранный период.	Сред ний

R-07	F	Система должна сохранять данные между запусками программы.	Высокий
R-08	F	Система должна выводить понятные сообщения при ошибочном вводе.	Средний

Основной пользовательский сценарий выглядит так:

1. Пользователь запускает программу.
2. Программа загружает задачи и журнал времени из файлов.
3. Пользователь создает задачу или выбирает уже существующую.
4. Пользователь запускает таймер.
5. После завершения работы пользователь останавливает таймер.
6. Программа сохраняет запись времени и показывает рассчитанную длительность.
7. Пользователь открывает отчет и видит суммарное время по задаче.

2.4 UML-диаграммы

Для документирования проекта подготовлены три UML-диаграммы. Диаграмма вариантов использования показывает, какие действия доступны пользователю. Диаграмма классов фиксирует будущую структуру C++ кода. Диаграмма последовательности описывает один ключевой сценарий — запуск и остановку таймера.

Диаграмма вариантов использования: тайм-трекер



Рисунок 2.1 — Диаграмма вариантов использования

Диаграмма классов: проектная структура C++

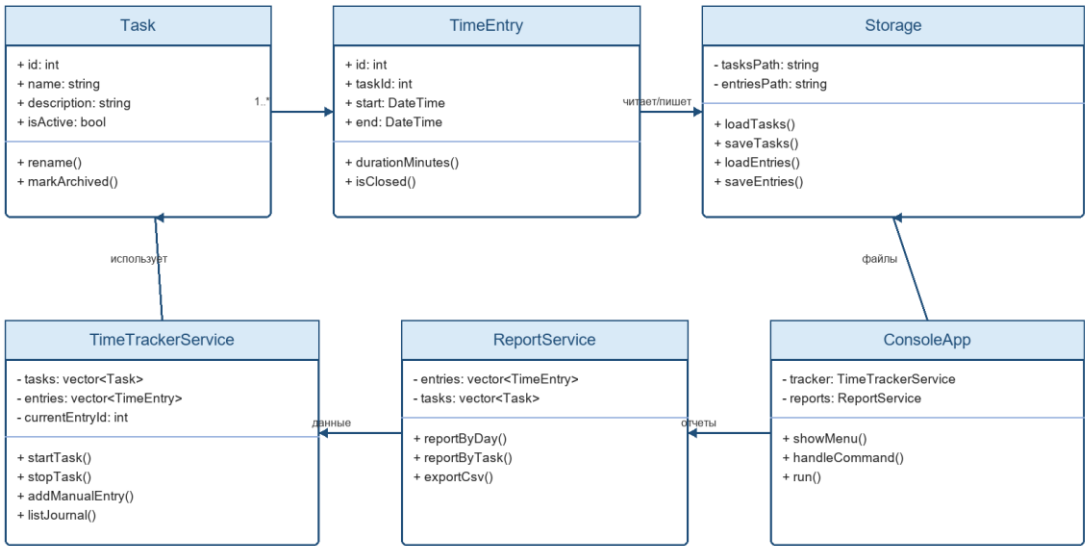
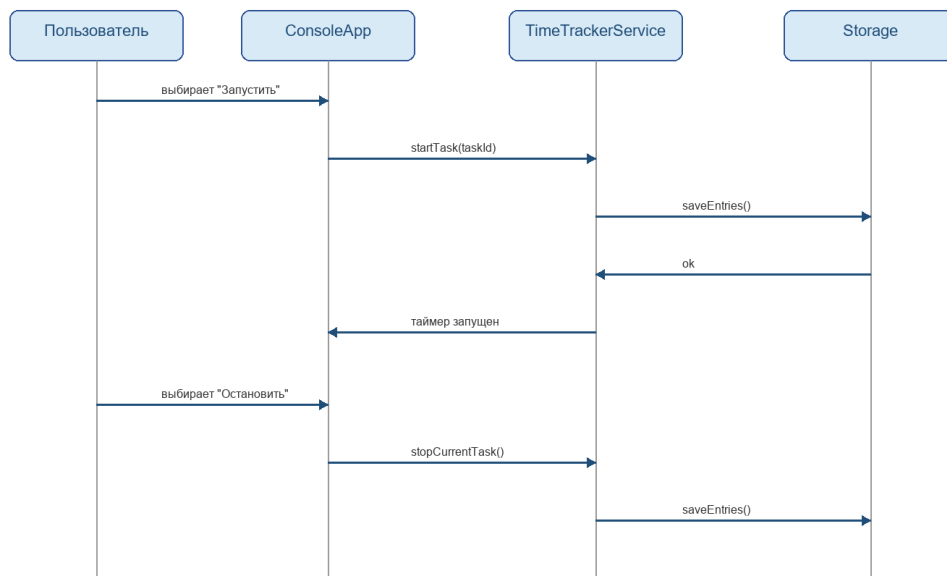


Рисунок 2.2 — Диаграмма классов

Диаграмма последовательности: запуск и остановка таймера



Основной результат: в журнале появляется закрытая запись времени с началом, окончанием и длительностью.

Рисунок 2.3 — Диаграмма последовательности

3 ПРАКТИЧЕСКАЯ ЧАСТЬ

3.1 План пользовательского интерфейса

Так как будущая реализация должна быть понятной и быстрой, для первой версии выбирается консольный интерфейс. Он состоит из главного меню и отдельных экранов ввода. Пользователь не запоминает команды, а выбирает действие по номеру.

Пункт меню	Назначение
1. Задачи	Создание, просмотр и изменение списка задач.
2. Запустить таймер	Начать учет времени по выбранной задаче.
3. Остановить таймер	Закрыть текущую запись времени и сохранить длительность.
4. Журнал	Показать последние записи времени.
5. Отчеты	Вывести суммарное время по задачам или дням.
0. Выход	Сохранить данные и завершить программу.

3.2 План структуры C++ проекта

Проект удобно разделить на несколько файлов. Это позволит не складывать весь код в `main.cpp`, а объяснить каждую часть отдельно. Для защиты это важно: структура проекта показывает, что программа не является набором случайных функций.

Файл	Назначение
main.cpp	Точка входа: создание приложения и запуск главного меню.
Task.h / Task.cpp	Класс задачи: идентификатор, название, описание, признак архивирования.
TimeEntry.h / TimeEntry.cpp	Класс записи времени: задача, начало, окончание, расчет длительности.
Storage.h / Storage.cpp	Загрузка и сохранение CSV-файлов.
TimeTrackerService.h / .cpp	Основная логика запуска, остановки и ручного добавления записей.
ReportService.h / .cpp	Суммирование времени и подготовка отчетов.
ConsoleApp.h / .cpp	Меню, ввод пользователя и вывод результатов.

Для первой версии достаточно стандартной библиотеки C++. Планируется использовать `vector` — динамический список объектов, `string` — строковый тип, `fstream` — чтение и запись файлов, `chrono` — работа со временем.

3.3 План тестирования

Тестирование первой версии можно выполнить вручную по контрольным сценариям. Автоматические тесты полезны, но для учебной защиты достаточно показать, что основные операции работают и ошибки обрабатываются предсказуемо.

Проверка	Ожидаемый результат
Создание задачи с нормальным названием	Задача появляется в списке и получает идентификатор.
Создание задачи с пустым названием	Программа выводит ошибку и не создает запись.
Запуск таймера	Появляется активная запись времени.
Повторный запуск таймера	Программа сообщает, что таймер уже запущен.
Остановка таймера	Запись закрывается, длительность рассчитывается автоматически.
Перезапуск программы	Ранее сохраненные задачи и записи загружаются из файлов.
Формирование отчета	Суммарное время совпадает с данными журнала.

3.4 Инструкция по эксплуатации

Пользователь запускает исполняемый файл программы. После запуска появляется главное меню. Для учета времени нужно сначала создать задачу, затем выбрать команду запуска таймера. После завершения работы пользователь выбирает команду остановки таймера. Программа сохраняет запись и показывает длительность.

Если пользователь забыл включить таймер, он может добавить запись вручную: выбрать задачу, указать дату, время начала и время окончания. При формировании отчета пользователь выбирает период, после чего программа выводит суммарное время по задачам.

ЗАКЛЮЧЕНИЕ

В работе выбрана тема будущего проекта — тайм-трекер для учета рабочего времени. Выбор обоснован простотой предметной области и реалистичностью реализации на C++. Для проекта собраны и

классифицированы требования по Вигерсу, подготовлена SRS-документация, описаны пользовательские сценарии и будущая структура программы.

Главное преимущество выбранной темы — она не требует сложных внешних технологий. Минимальная версия может быть реализована через консольное меню, классы, списки объектов и файлы. Поэтому проект подходит для студента, которому важно не утонуть в технических деталях, а спокойно объяснить логику работы программы на защите.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

8. Практическое занятие №2 «Программирование корпоративных систем» (ЭФБО-01,02,03,04,05,06-24).
9. Темы проектов (ЭФБО-01,02,03,04,05,06-24).
10. ОТЧЕТ ДЛЯ ПРАВОК НОВЫЙ — пример структуры и оформления отчета.
11. Практическое занятие №1 «Программирование корпоративных систем» — пример начального C++ проекта.

ПРИЛОЖЕНИЯ

Приложение А — Пояснительная карта проекта

Поле	Описание
Название проекта	Тайм-трекер (учет рабочего времени).
Целевая аудитория	Студенты, фрилансеры, начинающие разработчики, которым нужен простой учет времени.
Минимальная версия	Консольное приложение с задачами, таймером, журналом и отчетом.
Язык реализации	C++.
Хранение данных	Локальные CSV-файлы.
Главный риск	Ошибки при вводе даты и времени, которые нужно обрабатывать проверками.

Приложение Б — Краткий бэклог

Элемент	Описание	Статус для первой версии
Создание задач	Пользователь добавляет задачи для учета времени.	Обязательно
Таймер	Запуск и остановка текущей работы.	Обязательно
Журнал	Просмотр всех записей времени.	Обязательно
Отчет по задачам	Суммирование времени по каждой задаче.	Обязательно
Экспорт отчета	Сохранение отчета в CSV.	Желательно
Графический интерфейс	Окна, кнопки и визуальные отчеты.	Не входит в первую версию

4 ПАТТЕРНЫ ПРОЕКТИРОВАНИЯ

В практическом занятии №3 проект тайм-трекера рассматривается уже не только как набор требований, а как будущая C++ программа с понятными границами ответственности. Паттерн проектирования — это типовое решение часто повторяющейся архитектурной задачи. В данном проекте паттерны нужны не для усложнения, а для того, чтобы код было проще объяснить, тестировать и расширять.

Для учебной реализации выбраны простые и защищаемые паттерны: Facade для общего сервиса тайм-трекера, Repository для файлового хранилища, Strategy для отчетов, Factory Method для создания объектов и Singleton как пример паттерна, который лучше не применять без необходимости.

4.1 Применимые паттерны

Паттерн	Тип	Где применяется	Зачем нужен
Facade	Структурный	TimeTrackerService	Скрывает детали работы с задачами и записями времени за простыми методами startTask и stopCurrentTask.
Repository	Структурный	Storage	Отделяет бизнес-логику от чтения и записи CSV-файлов.
Strategy	Поведенческий	ReportService	Позволяет в будущем добавлять разные виды отчетов без переписывания основной логики.
Factory Method	Порождающий	createTask и создание TimeEntry	Сосредотачивает правила создания объектов в одном месте.
Singleton	Порождающий	Не используется в первой версии	Может пригодиться для глобальной конфигурации, но в учебном проекте усложнит тестирование.

4.2 Facade на примере TimeTrackerService

Facade переводится как «фасад». Смысл паттерна в том, что внешний код работает с простым интерфейсом, а сложные внутренние действия остаются внутри класса. В тайм-трекере таким фасадом является TimeTrackerService. Пользовательский интерфейс не должен вручную искать задачу, создавать TimeEntry, проверять активный таймер и считать следующий идентификатор. Он просто вызывает startTask или stopCurrentTask.

```
tracker.startTask(taskId);
TimeEntry closed = tracker.stopCurrentTask();
```

При защите это можно объяснить очень просто: фасад — это как стойка регистрации. Пользователь говорит, что ему нужно, а внутренние действия выполняет система.

4.3 Repository на примере Storage

Repository — это слой доступа к данным. В проекте он представлен классом Storage. Классы Task, TimeEntry и TimeTrackerService не должны знать, как именно данные записываются в файл: через CSV, JSON или другой формат. Они работают с объектами, а Storage отвечает за преобразование объектов в строки файла и обратно.

Метод Storage	Назначение
loadTasks	Загружает список задач из файла tasks.csv.
saveTasks	Сохраняет список задач в файл tasks.csv.
loadEntries	Загружает журнал записей времени из entries.csv.
saveEntries	Сохраняет журнал записей времени в entries.csv.

4.4 Strategy на примере отчетов

Strategy означает «стратегия». Этот паттерн полезен, когда одну задачу можно выполнять разными способами. В тайм-трекере это отчеты: отчет по задачам, отчет по дням, отчет за период, экспорт в CSV. В текущей версии

реализованы отчеты `minutesByTask` и `minutesByDay`, а выделение `ReportService` оставляет место для новых стратегий отчетности: по периоду, по активным задачам или с экспортом в CSV.

В дальнейшем можно сделать интерфейс `IReportStrategy` с методом `buildReport`, а конкретные классы `TaskReportStrategy` и `DayReportStrategy` будут реализовывать разные способы группировки данных. Это позволит добавлять новые отчеты без изменения `TimeTrackerService`.

4.5 Factory Method и отказ от лишнего Singleton

Factory Method — это способ создавать объекты через отдельный метод, в котором собраны правила создания. В проекте таким методом является `createTask`: он сам вычисляет следующий идентификатор и проверяет корректность названия задачи. Для записей времени аналогичная логика находится в `startTask` и `addManualEntry`.

Singleton, наоборот, в первой версии лучше не применять. Например, можно было бы сделать глобальный объект `Storage` или `Config`, но тогда код сложнее тестировать: тесты начинают зависеть от общего состояния. Поэтому решение честное и простое — нужные объекты создаются явно и передаются туда, где они нужны.

5 СИСТЕМА СБОРКИ И СТРУКТУРА C++ ПРОЕКТА

Вторая часть практического занятия №3 посвящена промышленной организации C++ проекта. Для тайм-трекера создана структура с отдельными папками `src`, `include` и `tests`. Сборка выполняется через CMake — систему, которая генерирует проект под конкретный компилятор и позволяет собирать программу одинаковыми командами на разных платформах.

5.1 Файловая структура

Путь	Назначение
<code>include/Task.h</code>	Описание класса задачи.
<code>include/TimeEntry.h</code>	Описание записи времени.
<code>include/TimeTrackerService.h</code>	Фасад основной бизнес-логики.
<code>include/ReportService.h</code>	Интерфейс формирования отчетов.
<code>include/Storage.h</code>	Интерфейс файлового хранилища.
<code>src/*.cpp</code>	Реализация классов проекта.
<code>tests/timetracker_tests.cpp</code>	Проверки длительности, запрета второго таймера и отчета.
<code>CMakeLists.txt</code>	Корневой сценарий сборки проекта.
<code>README.md</code>	Краткое описание, команды сборки и запуска.

5.2 Настройка CMake

Файл `CMakeLists.txt` задает минимальную версию CMake 3.10, имя проекта, стандарт C++17, основной исполняемый файл `timetracker` и тестовый исполняемый файл `timetracker_tests`. Заголовочные файлы подключаются через `target_include_directories`.

```
cmake_minimum_required(VERSION 3.10)
project(TimeTrackerPractice LANGUAGES CXX)
set(CMAKE_CXX_STANDARD 17)
```

```
add_executable(timetracker ...)  
target_include_directories(timetracker PRIVATE include)
```

5.3 Команды сборки и запуска

Проект собирается во внешней директории build. Такой способ называется out-of-source build: временные файлы сборки не смешиваются с исходным кодом.

```
cmake -S . -B build  
cmake --build build  
.\build\Debug\timetracker.exe --demo  
.\build\Debug\timetracker_tests.exe
```

В ходе проверки проект был успешно сконфигурирован, собран и запущен. Тестовый исполняемый файл вывел сообщение `timetracker tests passed`. Дополнительно проверен сценарий через консольное меню: создание задачи, ручная запись времени, отчет по задачам и отчет по дням.

5.4 Связь структуры с паттернами

Разбиение на файлы сделано не формально. Каждая группа классов соответствует своей ответственности: модели хранят данные, сервис тайм-трекера управляет сценариями, Storage отвечает за файлы, ReportService готовит отчеты, `main.cpp` только запускает прототип. Благодаря этому проект проще расширять: например, можно заменить CSV-хранение на SQLite, почти не трогая код учета времени.

6 ФАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ ПРОЕКТА

После замечания о том, что проект должен соответствовать выбранной теме, реализация была доведена до полноценной консольной версии тайм-трекера. Теперь программа не ограничивается демонстрационным запуском: пользователь может работать через меню, создавать задачи, фиксировать время и получать отчеты.

6.1 Реализованные возможности

Возможность	Как реализована в коде
Создание задачи	ConsoleApp вызывает TimeTrackerService::createTask, объект Task получает id, название и описание.
Запуск таймера	Команда меню вызывает TimeTrackerService::startTask, при этом запрещается второй активный таймер.
Остановка таймера	TimeTrackerService::stopCurrentTask закрывает текущую TimeEntry и считает длительность.
Ручная запись времени	ConsoleApp принимает даты в формате YYYY-MM-DD HH:MM, DateTimeUtils преобразует строку во время.
Журнал записей	ConsoleApp выводит список TimeEntry с началом, окончанием и длительностью.
Отчет по задачам	ReportService::minutesByTask суммирует минуты по названиям задач.
Отчет по дням	ReportService::minutesByDay группирует записи по дате начала.
Сохранение данных	Storage сохраняет задачи в tasks.csv, записи времени в entries.csv.

6.2 Фактическая структура файлов

Файл	Роль в проекте
include/ConsoleApp.h, src/ConsoleApp.cpp	Консольное меню и пользовательские сценарии.
include/Task.h, src/Task.cpp	Модель задачи: id, название, описание, активность.

include/TimeEntry.h, src/TimeEntry.cpp	Модель записи времени и расчет длительности.
include/TimeTrackerService.h , src/TimeTrackerService.cpp	Основная бизнес-логика тайм-трекера.
include/ReportService.h, src/ReportService.cpp	Отчеты по задачам и по дням.
include/Storage.h, src/Storage.cpp	Загрузка и сохранение CSV-файлов.
include/DateTimeUtils.h, src/DateTimeUtils.cpp	Преобразование даты и времени между строкой и объектом времени.
tests/timetracker_tests.cpp	Проверки ключевых сценариев без сторонних библиотек.
CMakeLists.txt	Сборка основного приложения timetracker и тестов timetracker_tests.
README.md	Инструкция по сборке, запуску и работе с меню.

6.3 Главное меню программы

При запуске без параметров приложение открывает меню. Все действия выполняются вводом номера команды, поэтому программу можно защитить и показать без графического интерфейса.

1. Create task
2. List tasks
3. Start timer
4. Stop timer
5. Add manual entry
6. Show journal
7. Report by task
8. Report by day
9. Save
0. Exit

6.4 Соответствие задания и реализации

Требование темы	Выполнение
Отметка начала задачи	Реализована командой Start timer через TimeTrackerService::startTask.
Отметка конца задачи	Реализована командой Stop timer через

	TimeTrackerService::stopCurrentTask.
Формирование отчетов	Реализованы отчеты Report by task и Report by day.
Реализация на C++	Проект написан на C++17 и использует стандартную библиотеку.
Разбиение на модули	Код разделен на пары .h/.cpp в include и src.
Сборка через CMake	CMakeLists.txt создает timetracker и timetracker_tests.
README	README.md содержит описание проекта, команды сборки и пример запуска.

6.5 Проверка

Проверка выполняется через CMake и тестовый исполняемый файл. Тесты покрывают расчет длительности, запрет второго таймера, архивирование задачи, ручную запись времени, отчеты, сохранение и загрузку, а также сценарий работы через ConsoleApp.

```
cmake -S . -B build
cmake --build build
.\build\Debug\timetracker_tests.exe
.\build\Debug\timetracker.exe --demo
```

7 ТЕСТИРОВАНИЕ ПРОЕКТА

Практическое занятие №4 посвящено тестированию проекта. Для тайм-трекера была подключена библиотека Catch2 и подготовлена система unit-тестов для основных классов, а также отдельные сценарные мини-программы, которые проверяют реальные пользовательские маршруты работы с приложением.

7.1 Подключение Catch2

Для тестирования выбрана библиотека Catch2. Она удобна для учебного проекта, потому что позволяет писать читаемые тесты через макросы TEST_CASE и REQUIRE. Заголовочный файл библиотеки расположен в папке external/catch2, а тестовая цель подключает эту папку через target_include_directories.

```
add_executable(timetracker_tests tests/unit_tests.cpp)
target_include_directories(timetracker_tests PRIVATE external/catch2)
configure_test_target(timetracker_tests)
add_test(NAME unit_tests COMMAND timetracker_tests)
```

7.2 Структура тестовой части

Файл	Назначение
external/catch2/catch.hpp	Локально подключенная заголовочная версия Catch2.
tests/unit_tests.cpp	Unit-тесты классов проекта.
tests/scenarios/scenario_basic_workflow.cpp	Мини-программа базового сценария: задача, таймер, отчет.
tests/scenarios/scenario_persistence.cpp	Мини-программа проверки сохранения и загрузки CSV.
tests/scenarios/scenario_console_workflow.cpp	Мини-программа сценария работы через ConsoleApp.
CMakeLists.txt	Регистрация тестовых целей и CTest-сценариев.

7.3 Unit-тесты по классам

Unit-тест проверяет небольшой участок программы отдельно от остальных. В проекте тесты написаны для каждого класса, участвующего в основной логике: моделей, сервисов, хранилища, утилит даты и консольного приложения.

Класс	Что проверяется
Task	Создание задачи, проверка id, названия, описания, активности, переименование, архивирование, ошибки при пустом названии и неверном id.
TimeEntry	Открытая и закрытая запись времени, расчет длительности, сохранение начала и конца, запрет отрицательного интервала.
TimeTrackerService	Создание задач, последовательные id, запуск и остановка таймера, запрет второго таймера, ручная запись, переименование, архивирование, ошибка неизвестной задачи.
ReportService	Суммирование минут по задачам, группировка по дням, игнорирование незакрытых записей, обработка неизвестной задачи.
DateTimeUtils	Преобразование строки в дату и обратно, формат даты, формат даты и времени, ошибки неверного формата.
Storage	Пустые файлы, сохранение и загрузка задач, сохранение архивации, сохранение и загрузка записей времени.
ConsoleApp	Полный сценарий через ввод пользователя: создание задачи, ручная запись, журнал, отчет по задачам, отчет по дням, сохранение.

7.4 Тестовые сценарии

Кроме unit-тестов добавлены отдельные мини-программы. Они проверяют не отдельный метод, а связанный пользовательский сценарий. Такой подход полезен, потому что часть ошибок проявляется только при совместной работе нескольких классов.

Сценарий	Проверяемый маршрут	Ожидаемый результат
scenario_basic_workflow	Создать задачу, запустить таймер, остановить таймер, сформировать отчет.	Отчет показывает 60 минут по задаче.

scenario_persistence	Сохранить задачу и запись времени, затем загрузить их из CSV.	После загрузки есть 1 задача и 1 запись на 30 минут.
scenario_console_workflow	Пройти сценарий через ConsoleApp с заранее подготовленным вводом.	Отчет по задаче показывает 75 минут.

7.5 Запуск тестов

Все тесты зарегистрированы в CTest. Так как на данной машине используется генератор Visual Studio, при запуске CTest нужно указать конфигурацию Debug.

```
cmake -S . -B build
cmake --build build
ctest --test-dir build -C Debug --output-on-failure
```

Тестовые исполняемые файлы собираются в отдельную директорию build/tests: timetracker_tests, scenario_basic_workflow, scenario_persistence и scenario_console_workflow.

7.6 Результаты проверки

Проверка	Результат
Конфигурация CMake	Выполнена успешно командой cmake -S . -B build.
Сборка проекта	Выполнена успешно командой cmake --build build.
Unit-тесты Catch2	Цель unit_tests прошла успешно.
Сценарий basic workflow	Сценарий прошел успешно.
Сценарий persistence	Сценарий прошел успешно.
Сценарий console workflow	Сценарий прошел успешно.
Итог CTest	100% tests passed, 0 tests failed out of 4.

7.7 Вывод по тестированию

В результате практического занятия проект получил полноценную тестовую основу. Unit-тесты проверяют основные классы и функции, а

сценарные программы подтверждают, что части проекта работают вместе. Это делает дальнейшее развитие тайм-трекера безопаснее: при изменении кода можно быстро проверить, что создание задач, учет времени, отчеты и сохранение данных не сломались.